Software Performance Engineering



Quiz 2 Practice

Atharva Kirkole

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

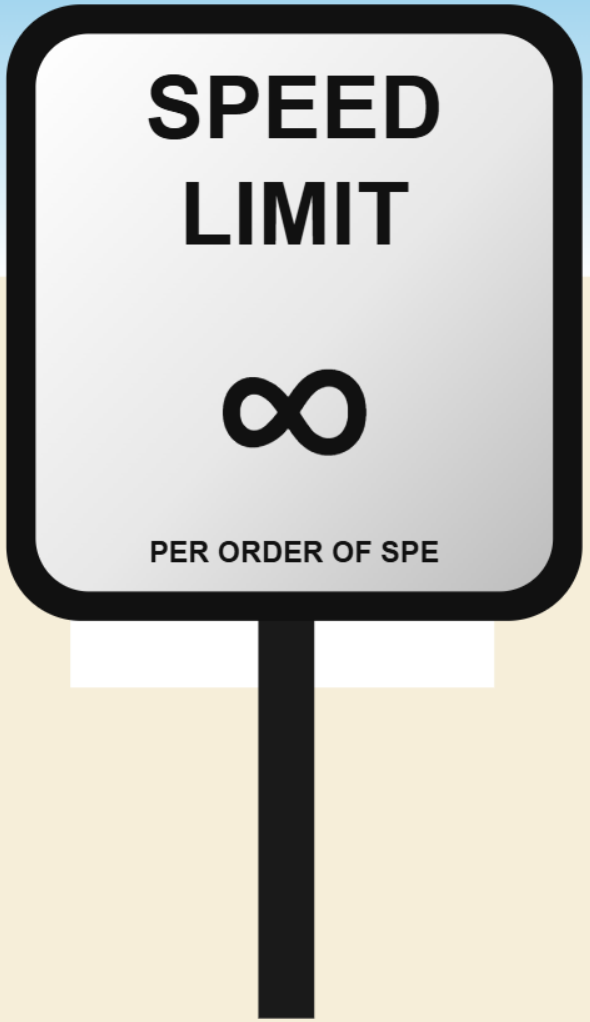
Due Dates

Project 2 Final: 15th November

HW 5 : 16th November

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

MULTIPLE CHOICE



© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 1: Scheduling bounds

- A Cilk program has work $T_1 = 2048$ and span $T\infty = 16$.
- You run it on P = 32 processors under a **greedy** scheduler.
- Which of these is guaranteed about the running time T_{32}?

A. $T_{32} = 64$ exactly
B. $16 \leq T_{32} \leq 64$
C. $64 \leq T_{32} \leq 80$
D. $64 \leq T_{32} \leq 2048$
E. None of the above

- [3 points]

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 1: Answer

The question

- $T_1 = 2048$, $T_\infty = 16$, $P = 32$.
- Greedy scheduler. Guaranteed bound on T_{32}?

A. $T_{32} = 64$ exactly
B. $16 \le T_{32} \le 64$
C. $64 \le T_{32} \le 80$
D. $64 \le T_{32} \le 2048$
E. None

Answer: C

- Two **lower** bounds hold for any scheduler:
  - the work law $T_3 \ge T_1/P = 2048/32 =$ **64**
  - the span law $T_3 \ge T_\infty = 16$
  - so $T_{32} \ge \max(64, 16) = 64$
- Greedy's **upper** bound is $T_3 \le T_1/P + T_\infty$:
  - $64 + 16 =$ **80**
- So $64 \le T_{32} \le 80$. **C.**
- A says exactly 64: that would need parallelism to be perfect. not guaranteed.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 2: Nested parallel loops

```
cilk_for (int i = 1; i < n; i++)
 cilk_for (int j = 0; j < i; j++)
  B[i][j] = g(A[i], A[j]);
```

- g costs Θ(1). Both loops use the usual recursive
- What is the **span** of this computation?

  A. $\Theta(\log n)$
  B. $\Theta(\log^2 n)$
  C. $\Theta(n)$
  D. $\Theta(n \log n)$
  E. None of the above

- [3 points]

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 2: Answer

The question

```
cilk_for (i = 1; i < n; i++)
 cilk_for (j = 0; j < i; j++)
  B[i][j] = g(A[i], A[j]);
```

- g is Θ(1), grainsize 1. Span?

  A. Θ(log n)
  B. Θ(log² n)
  C. Θ(n)
  D. Θ(n log n)
  E. None

Answer: A

- A cilk_for over m iterations with a Θ(1) body is a balanced binary spawn tree: span **Θ(log m)**.
- The critical path enters the outer tree, Θ(log n), picks *one* value of i, and runs that inner tree, Θ(log i).
  - the other n-1 inner loops are in parallel and cost nothing on the path

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 3: Work stealing

- A Cilk computation with work T_1 and span $T\infty$ runs under the
  - **randomized work-stealing** scheduler on P workers.
- In expectation, how many **successful steals** occur?

  A. $O(T_1/P)$
  B. $O(P\ T\infty)$
  C. $O(T\infty)$
  D. $O(P)$
  E. None of the above

- [3 points]

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Problem 3: Answer

The question

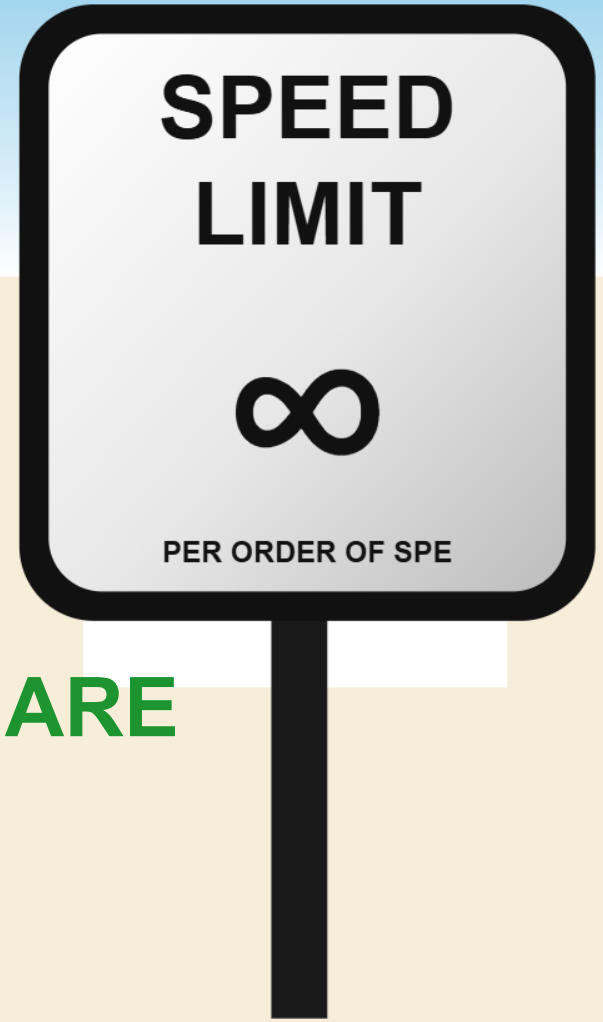
- Randomized work stealing, P workers, work T_1, span $T\infty$.
- Expected number of successful steals?

A. $O(T_1/P)$
B. $O(P\ T\infty)$
C. $O(T\infty)$
D. $O(P)$
E. None

Answer: B

- Every worker is at every moment either **working** or **stealing**. That is the whole accounting argument.
- Work tokens total T_1, so work time is exactly T_1/P per worker on average. The steals are the other bucket.
- Each round of P steal attempts reduces the remaining span by a constant factor in expectation, so **$O(T\infty)$** rounds suffice, each costing O(P) attempts: **$O(P\ T\infty)$** steals.

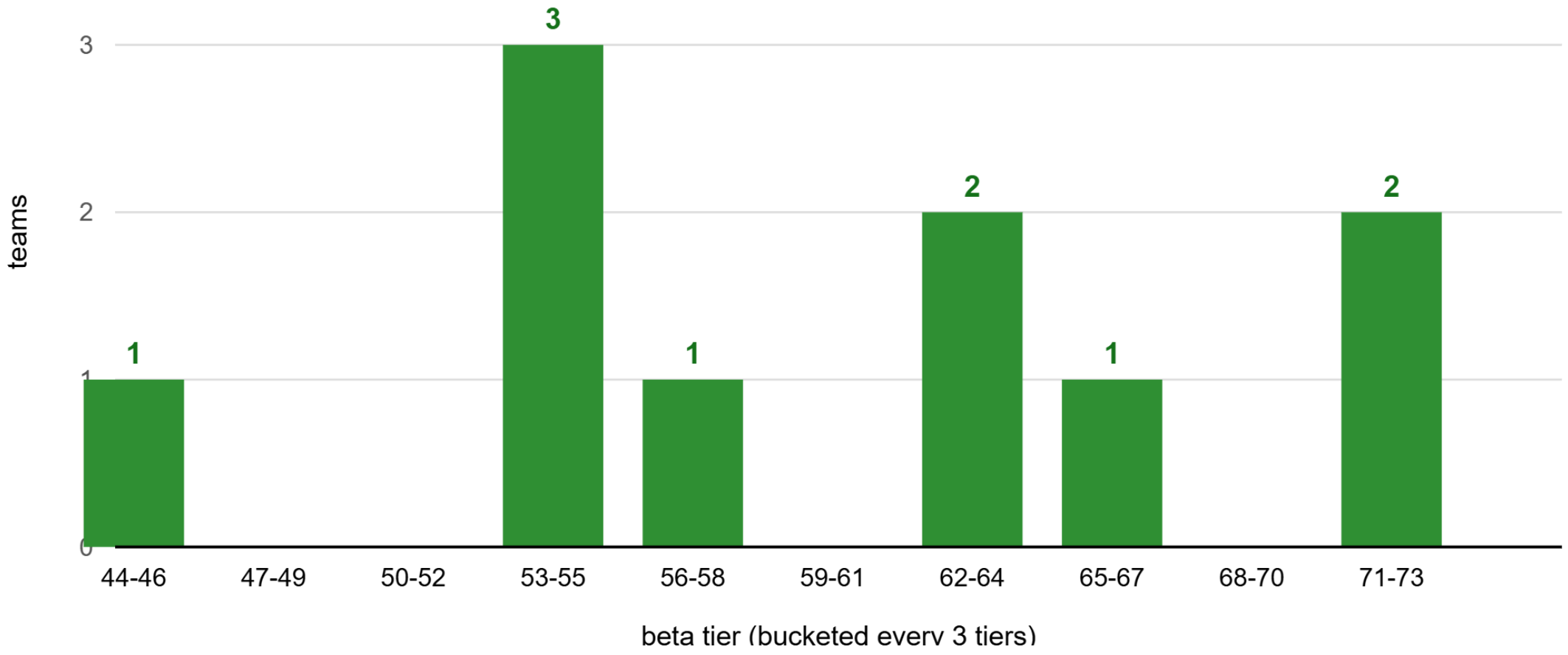
© 2008–2024 by the MIT 6.172 and 6.106 Lecturers



PROJECT 2: WHERE THE TIERS ARE

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Where everyone is right now



10 teams · **mean 60.0** **median 59.5** range **44 to 72**

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

Make the survivors cheap too

The expensive check

```
double dx = px[j] - px[i];
double dy = py[j] - py[i];
double dz = pz[j] - pz[i];
double d = sqrt(dx*dx+dy*dy+dz*dz);
if (d < r[i] + r[j]) {
 /* ... solve for t ... */
}
```

- sqrt is tens of cycles and it is on the path of *every* pair that survived the sweep.

Three rejects before you pay

```
// 1. never sqrt to compare a distance
double rsum = r[i] + r[j];
double d2 = dx*dx + dy*dy + dz*dz;
if (d2 > rsum * rsum) continue;

// 2. finish the box test on the OTHER axes,
// which you skipped during the x sweep
if (ylo[j] > yhi[i] || yhi[j] < ylo[i]) continue;
if (zlo[j] > zhi[i] || zhi[j] < zlo[i]) continue;

// 3. cannot beat what you already found
if (t_earliest_possible >= best.t) continue;

check_for_collision(sph, i, j, &best);
```

– order them cheapest-first; a reject that costs more than the thing it is avoiding is not a reject

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers

- The questions that separate the mid 60s from the 70-80s, which
  - are yours to answer:
  - **which axis** you sort on, and whether you re-sort or repair
  - the order between timesteps, the list is nearly sorted already
  - **what survives a ministep.** Which parts of your candidate set
  - are still valid, and how you detect the ones that are not
  - loop stops dominating, something else does. Profile again.

© 2008–2024 by the MIT 6.172 and 6.106 Lecturers